

Day 16 Pandas Part 2

Gemini said

Here are your notes based on the presentation, complete with practical Python code examples for every topic covered.

Concatenation - Stacking Data

Concatenation uses `pd.concat()` to "glue" DataFrames together, either vertically by adding rows or horizontally by adding columns. This is useful for unifying datasets that share the same structure but are split across different files or time periods, like combining 12 separate monthly sales CSVs into a yearly report.

+3

```
import pandas as pd

# Example: Appending new log entries to a master log
master_log = pd.DataFrame({'Log_ID': [1, 2], 'Message': ['System Start', 'User Login']})
new_logs = pd.DataFrame({'Log_ID': [3, 4], 'Message': ['Data Processed', 'System Shutdown']})

# Vertically stacking the DataFrames
combined_logs = pd.concat([master_log, new_logs], ignore_index=True)
print(combined_logs)
```

Merging & Joining - Linking Data

Merging uses `pd.merge()` to connect two different tables based on a shared "Key" (such as a User_ID). Because databases are relational, information is often split; for instance, user details in one table and their orders in another.

+2

- **Inner Join:** Keeps only records found in both tables.

- **Outer Join:** Keeps all records from both, filling gaps with NaN.
- **Left Join:** Keeps everything from the "Master" table and adds info from the "Detail" table where it exists.

```
customers = pd.DataFrame({'User_ID': [101, 102], 'Name': ['Alice', 'Bob']})
transactions = pd.DataFrame({'User_ID': [101, 101, 103], 'Amount': [250, 100, 400]})

# Inner Join: Links Customers with Transactions based on User_ID
# (Notice User_ID 103 is dropped because it's not in the customers table)
merged_data = pd.merge(customers, transactions, on='User_ID', how='inner')
print(merged_data)
```

GroupBy - The Pivot Engine

GroupBy is a process that involves three steps: Split (grouping by a category), Apply (calculating a metric like sum or average), and Combine (showing the result). This helps find patterns and summaries within large datasets, such as finding the average salary per department.

+2

```
data = pd.DataFrame({
    'Department': ['IT', 'HR', 'IT', 'HR', 'IT'],
    'Salary': [70000, 50000, 80000, 55000, 75000]
})

# Finding the Average Salary per Department
avg_salary = data.groupby('Department')['Salary'].mean()
print(avg_salary)
```

Discretization & Binning

Discretization uses methods like `pd.cut()` or `pd.qcut()` to turn continuous numbers into "Buckets" or "Categories". This simplifies numerical data for better decision-making or creates categorical labels for machine learning, like converting test scores (0–100) into grades (A, B, C, F).

+2

```
students = pd.DataFrame({'Name': ['Ali', 'Zara', 'Hassan'], 'Score': [88, 55, 72]})

# Converting "Test Scores" into "Grades"
bins = [0, 60, 75, 90, 100]
labels = ['F', 'C', 'B', 'A']
students['Grade'] = pd.cut(students['Score'], bins=bins, labels=labels)
print(students)
```

Operations & Data Output

This involves final manipulations like creating a `.pivot_table()` and saving your processed data using `.to_csv()` or `.to_excel()`. Exporting data allows non-developers to open and read your cleaned reports.

+1

```
# Creating a simple pivot table
sales = pd.DataFrame({'City': ['LHR', 'KHI', 'LHR'], 'Sales': [100, 200, 150]})
pivot = sales.pivot_table(index='City', values='Sales', aggfunc='sum')
print(pivot)

# Exporting a cleaned report to a CSV file
pivot.to_csv('weekly_inventory_report.csv')
```

Plotting – Distributions (Hist, Box, KDE)

Pandas has built-in plotting to visualize how data is spread out. This is highly useful for spotting "Outliers" (errors) and understanding the "Typical" value in a dataset, such as using a histogram to

see which hours of the day generate the most website traffic.

+3

```
import matplotlib.pyplot as plt

traffic_data = pd.DataFrame({'Traffic_Hours': [1, 2, 2, 3, 3, 3, 4, 4,
5, 18, 19]})

# Histogram to see the distribution of traffic
traffic_data['Traffic_Hours'].plot.hist(bins=10, title="Website Traffic
Distribution")
plt.show()
```

Plotting – Relationships (Scatter, Line, Bar)

Relationship plots visualize how one variable affects another to identify trends over time or correlations. A common real-life usage is a scatter plot to see if the "Time spent on App" correlates with the "Total Money Spent".

+2

```
app_data = pd.DataFrame({
    'Time_Spent_Mins': [10, 20, 30, 40, 50],
    'Money_Spent_USD': [5, 15, 25, 45, 60]
})

# Scatter Plot to see the correlation between time and money spent
app_data.plot.scatter(x='Time_Spent_Mins', y='Money_Spent_USD',
title="Time vs Money Spent")
plt.show()
```

Would you like to try combining two of these concepts in a practice exercise, like joining two tables and then using GroupBy to summarize the results?

